

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

ROHIT L - 192511236

Descriptive Written Test: Intelligent Agents & Search Strategies

Comprehensive Solutions for Smart Home Agent Architectures and Maze Navigation Search Algorithms

Question 1: Smart Home Automation System Agent Design

Problem Context: A smart home automation system is designed to manage lighting, temperature, and security based on real-time inputs (motion detection, ambient temperature, time of day) while learning user preferences over time. We analyze the four primary intelligent agent architectures within this domain, evaluate their functionality, and justify an optimal hybrid agent architecture.

1. Overview of Intelligent Agent Architectures

An intelligent agent is an autonomous entity that perceives its environment through sensors, processes information, and acts upon the environment using actuators to achieve specific goals. In AI theory, agents are categorized into four structural types based on their internal decision-making complexity:

- **Simple Reflex Agents:** Act purely on condition-action rules (if condition $\Rightarrow$ action) mapped directly from the current percept, completely ignoring percept history.
- **Model-Based Reflex Agents:** Maintain an internal state to track unobserved aspects of the world, updating their state as new percepts arrive.
- **Goal-Based Agents:** Combine internal state tracking with explicit goal descriptions to plan sequence of actions that transition the environment into a desired state.
- **Utility-Based Agents:** Use a continuous utility function mapping states to real numbers, allowing explicit optimization across multiple trade-offs (e.g., comfort vs. energy cost).

2. Functional Analysis of Agent Types in Smart Home Automation

A. Simple Reflex Agent in Smart Home

Mechanism: Operates using a lookup table or hardcoded condition-action rules on instantaneous percepts.

- **Lighting:** IF MotionDetected = TRUE AND Time = Night THEN TurnOn(Lights).
- **Temperature:** IF Temperature > 24°C THEN TurnOn(AC).
- **Security:** IF DoorSensor = Tripped AND AlarmState = Armed THEN SoundAlarm().

Limitations: Lacks memory and context. If a user sits still reading a book at night, motion ceases; the simple reflex agent immediately extinguishes the lights, causing frustration. It cannot handle partially observable states or subtle preferences.

B. Model-Based Reflex Agent in Smart Home

Mechanism: Maintains an internal state model representing 'how the world works' (e.g., thermal inertia, room occupancy persistence, time elapsed since motion).

- **Internal State:** Tracks OccupancyState = Occupied for 15 minutes after motion stops. Tracks AC status knowing that cooling takes time to lower room temperature.
- **Lighting:** Keeps lights on while OccupancyState = Occupied, preventing premature shut-off.
- **Security:** Distinguishes between a family member walking around inside vs. an external perimeter breach by keeping track of entry point history.

Limitations: Fixed rules prevent adaptation to changing seasonal patterns or user mood; cannot resolve conflicting objectives (e.g., cooling the house when electricity tariffs spike).

C. Goal-Based Agent in Smart Home

Mechanism: Formulates multi-step action plans to satisfy explicit goals (e.g., Goal 1: 'Keep living room at 22°C by 6:00 PM', Goal 2: 'Secure perimeter by 10:00 PM').

- **Temperature & HVAC Planning:** The agent uses search and planning to evaluate: 'If I turn on the heat pump at 5:15 PM, ambient temperature will reach 22°C exactly when the resident arrives at 6:00 PM.'
- **Security Sequencing:** Locks doors, arms motion detectors, and lowers window blinds automatically when the 'Arm Away' goal is activated.

Limitations: Binary goal fulfillment (satisfied vs. unsatisfied). If a goal cannot be 100% achieved (e.g., extreme outdoor heat prevents reaching 22°C), a goal-based agent cannot trade off partial comfort for significant energy savings.

D. Utility-Based Agent in Smart Home

Mechanism: Evaluates world states using a utility function U(S) that quantifies degree of happiness, balancing competing sub-objectives:

$$U(S) = w_1 \cdot \text{Comfort}(S) + w_2 \cdot \text{Security}(S) - w_3 \cdot \text{EnergyCost}(S)$$

- **Trade-off Decision Making:** During peak electricity tariff hours (2 PM – 6 PM), the utility agent slightly increases AC setpoint from 21°C to 23°C. It calculates that a 5% minor reduction in comfort yields a 40% reduction in energy bill, maximizing total user utility.
- **Personalized Learning:** Learns preference weights w_i using reinforcement learning based on manual user overrides.

3. Comparative Evaluation Table of Smart Home Agent Types

Agent Type	Comfort Management	Energy Efficiency	Security Management
Simple Reflex	Poor; abrupt light/AC switching on/off without memory.	Moderate; turns off devices instantly but lacks optimization.	Fast reflex alarms; highly vulnerable to false positives.
Model-Based	Good; retains occupancy state & room thermal inertia.	Fair; follows fixed schedules without tariff awareness.	Robust; tracks entry sequence to filter sensor noise.
Goal-Based	High; proactively prepares room conditions for arrival.	Good; plans efficient device operation windows.	Strong; verifies all security subsystems reach goal state.

Utility-Based	Optimal; balances comfort against cost preferences.	Maximum; optimizes power usage vs. peak electricity pricing.	Optimal; prioritizes critical security alerts flexibly.
---------------	---	--	---

4. Justification of the Most Effective Architecture: Learning Utility-Based Hybrid Agent

Recommended System: Hybrid Architecture combining a Learning Utility-Based Deliberative Core with a Model-Based Reflex Layer.

Key Advantages:

- 1. Safety & Low Latency (Reflex Layer): Critical security breaches (smoke, glass break) bypass utility planning and trigger immediate alarms (<10ms).
- 2. Multi-Objective Optimization (Utility Core): Manages HVAC/lighting by dynamically trading off comfort against real-time electricity tariff rates.
- 3. Adaptation over Time (Learning Component): Adjusts utility weights based on manual resident overrides via reinforcement learning.

Question 2: Uninformed Search Strategies for Unknown Maze Navigation

Problem Context: A mobile robot is placed in an unknown maze and must find a path to the exit without prior geographical layout knowledge. We analyze how uninformed search strategies solve this problem, compare time/space complexity, link search to agent design, and recommend the most effective search-agent combination.

1. Application of Uninformed Search in Unknown Mazes

In an unknown maze without domain heuristics, the robot must rely on uninformed (blind) search strategies. The state-space component formulation includes:

- **State Space (S):** Grid coordinates (x, y) representing reachable positions in the maze.
- **Initial State (S0):** Robot's starting location (x0, y0).
- **Actions (A):** Movement operators {North, South, East, West}.
- **Goal Test:** CurrentPos == Exit.
- **Path Cost (g(n)):** Cumulative movement cost (step length, terrain friction, battery energy consumed).

2. Deep-Dive into Uniform Cost Search (UCS)

Mechanism: UCS (Dijkstra's algorithm on trees/graphs) expands nodes in order of increasing path cost $g(n)$ using a Priority Queue ordered by $g(n)$.

- **Unequal Step Costs:** If maze corridors have varying travel costs (e.g., smooth tile = 1 cost unit, rough gravel = 3 cost units), UCS guarantees finding the absolute minimum-cost path to the exit.
- **Search Behavior:** Expands uniformly in concentric 'cost contours' outward from the start position.
- **Time Complexity:** $O(b^{(1 + \lceil C^* / \epsilon \rceil)})$ where C^* is optimal path cost and ϵ is minimum edge cost.
- **Space Complexity:** $O(b^{(1 + \lceil C^* / \epsilon \rceil)})$ — High RAM demand; stores all open nodes and explored set in memory.

3. Deep-Dive into Iterative Deepening Search (IDS)

Mechanism: IDS combines the spatial efficiency of Depth-First Search (DFS) with the optimality of Breadth-First Search (BFS). It performs repeated Depth-Limited Searches (DLS) with incrementally increasing depth limits ($d = 0, 1, 2, 3, \dots$).

- **Search Behavior:** Re-visits top-level nodes on each iteration, but node re-computation overhead is small (ratio $b/(b-1)$).
- **Uniform Step Costs:** When all maze moves have unit cost ($c = 1$), depth equals cost, making IDS complete and optimal.
- **Time Complexity:** $O(b^d)$ where d is the goal depth.
- **Space Complexity:** $O(b \cdot d)$ — Minimal RAM demand; linear memory footprint along the active search path.

4. Trade-Off Comparison Table: UCS vs. IDS

Evaluation Criterion	Uniform Cost Search (UCS)	Iterative Deepening Search (IDS)
Completeness	Complete if edge costs $c \geq \epsilon > 0$.	Complete if branching factor b is finite.
Optimality	Optimal for arbitrary non-negative path costs.	Optimal if step costs are equal (unit cost). Sub-optimal for variable step costs.

Time Complexity	$O(b^{(1 + \lfloor C^* / \epsilon \rfloor)})$ — dependent on path cost.	$O(b^d)$ — dependent on goal depth d .
Space Complexity	$O(b^{(1 + \lfloor C^* / \epsilon \rfloor)})$ — High RAM demand.	$O(b \cdot d)$ — Minimal linear memory footprint.
Cost Awareness	Fully handles variable terrain costs and risks.	Treats all moves as having uniform depth/cost.
Embedded Suitability	Risk of memory exhaustion on large mazes.	Ideal for memory-constrained microcontrollers.

5. Link Between Intelligent Agent Design and Search Decision-Making

The choice of agent architecture dictates how search strategies are embedded and executed during maze exploration:

- **Simple Reflex Agent:** Cannot run search algorithms. Executes instantaneous wall-following (e.g., Right-Hand Rule). Fails in mazes with loops or detached inner walls.
- **Model-Based Agent (Online Search):** Stores explored cells in an internal occupancy grid map G_{known} . As sensors discover new corridors, the model updates and reruns pathfinding dynamically.
- **Goal-Based / Utility-Based Agent:** Evaluates step costs (battery usage, wheel slip risk) alongside distance, seamlessly feeding non-uniform path costs into Uniform Cost Search.

6. Proposed Optimal Combination & Justification

Recommended Combination: Online Model-Based Utility Agent combined with Uniform Cost Search (or D* Lite / Online UCS)

Justification Summary:

1. Internal Mapping (Model-Based): Maintains a 2D occupancy grid map of explored corridors to prevent infinite looping in dead-ends.
2. Energy & Hazard Trade-offs (Utility-Based): Evaluates non-uniform costs (slopes, battery usage, friction) using a utility function $U(\text{Path}) = -\text{TotalCost}(\text{Path})$.
3. Minimum-Cost Traversal (UCS): Handles variable terrain step costs where IDS fails, guaranteeing the minimum-energy route to the exit.